

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF ELECTRONICS, TELECOMMUNICATIONS AND INFORMATION
TECHNOLOGY

MICROPROCESSORS MINI-PROJECT

Real-Time RFID Authentication

Contents:

1. **Introduction:** Why I chose this topic and the problem I am solving.
2. **Theoretical Background:** The key concepts (RFID, Wi-Fi, HTTP) used in the project.
3. **Implementation:**
 - How I built the hardware.
 - How the code works (Logic & Time Control).
 - Connecting to the Cloud (Google Sheets).
4. **Conclusions & Bibliography**

1. The Theme

My project focuses on building an intelligent "Smart Door" system. Instead of just using a simple key or a standard code, I wanted to create a system that knows who is entering and when they are trying to enter.

The system uses an ESP8266 microcontroller and an RFID reader to scan user tags. It then checks the time online (using NTP) to decide if that specific person is allowed in at that specific hour.

Finally, it logs everything—successes and failures—to a Google Sheet so I can see a colourful, real-time history of who accessed the room.

We see smart locks everywhere now, but most basic DIY security systems are "dumb"—they either let you in or they don't. They rarely account for *context*.

Additionally, keeping track of who entered a building usually involves writing things down on paper or buying expensive software.

To solve this, I designed a system that acts as an automated security guard.

- **It verifies identity:** Using unique RFID cards.
- **It checks the schedule:** It connects to the internet to get the exact time, ensuring "Worker1" can only enter between 17:00 and 19:00.
- **It keeps records:** Every time someone taps a card, the system sends a report to the cloud.

2. THEORETICAL BACKGROUND

To make this project work, I had to combine three main technical concepts:

RFID Technology (ISO/IEC 14443)

For the physical "key," I used Radio Frequency Identification (RFID). The MFRC522 reader generates a small electromagnetic field. When the passive card (the tag) comes close, it gets powered up and sends its unique ID back to the reader. I used the SPI protocol to let my microcontroller talk to the reader fast enough to feel instant.

The IoT Brain: ESP8266

I chose the ESP8266 (NodeMCU) because it's a microcontroller with built-in WiFi. This is crucial for my project because the device needs to be online to check the time and send data to Google. It uses standard HTTP/HTTPS requests, effectively acting like a web browser that "visits" a specific Google link to send data.

Google Apps Script

Instead of setting up a complex server, I used Google's scripting language (based on JavaScript). This allows the Google Spreadsheet to act as a backend database. The script receives the data from my ESP8266 and processes it instantly.

3. IMPLEMENTATION

- **Hardware Architecture** The system is built using the ESP8266 NodeMCU as the central controller. It communicates with the MFRC522 RFID reader via the SPI protocol and controls a Servo Motor via PWM on pin D1. A buzzer on pin D8 provides audio feedback for user interactions. The device connects to the local WiFi network ("Barlog") to access NTP time services and Google Cloud APIs.
- **Software Logic** The firmware is written in C++ (Arduino). To keep the main loop efficient, the code performs three critical steps: identifying the user, validating the time, and logging the result.
 - a) **RFID Data Conversion** The system reads Block 2 of the RFID card. Since the raw data comes in bytes, the code must convert this into a readable string to identify the user (e.g., "Worker1").

Code Snippet: Converting Card Data to String

C++

```
// Converting the raw byte array into a String name
String rfid_name = "";

for (int j = 0; j < 16; j++){
    if (readBlockData[j] != 0x00) {
        rfid_name += (char)readBlockData[j];
    }
}

rfid_name.trim(); // Removes whitespace for accurate matching
```

- b) **Time-Based Access Control** The core innovation of this project is the logic that restricts specific users based on the time of day, retrieved via the NTPClient. The system checks if the user is "Worker1" and if the current hour is strictly between 17:00 and 19:00.

Code Snippet: The Decision Logic

C++

```
int currentHour = timeClient.getHours();

// "Boss" has unlimited access

if (rfid_name == "Boss") {

    openDoor = true;

}

// "Worker1" is restricted to 17:00 - 19:00

else if (rfid_name == "Worker1") {

    if (currentHour >= 17 && currentHour < 19) {

        openDoor = true;

    } else {

        openDoor = false; // Access denied outside shift

    }

}
```

- c) **Cloud Logging (HTTPS)** Regardless of whether access is granted or denied, the system logs the event. It constructs a secure URL containing the user's name and status, then sends a GET request to the Google Script.

Code Snippet: Sending Data to Google Sheets

C++

```
// Constructing the URL with parameters

String url = sheet_url + name + "&sts=" + status;

HTTPClient https;

// Sending the secure GET request

// Note: setInsecure() is used to handle SSL on ESP8266

client->setInsecure();

if (https.begin(*client, url)){

    https.GET();

    https.end();

}
```

d) Main Microcontroller Firmware (C++)

```
#include <SPI.h>

#include <MFRC522.h>

#include <Arduino.h>

#include <ESP8266WiFi.h>
```

```

#include <ESP8266HTTPClient.h>

#include <WiFiClient.h>

#include <WiFiClientSecureBearSSL.h> // Required for secure (HTTPS) connection to Google

#include <Servo.h>

#include <WiFiUdp.h>

#include <NTPClient.h>

// --- Hardware Pin Definitions ---

#define RST_PIN D2 // Reset pin for RFID

#define SS_PIN D4 // Slave Select pin for SPI communication

#define BUZZER D8 // Active Buzzer for audio feedback

#define SERVO_PIN D1 // Servo motor control pin

// --- Network Credentials ---

#define WIFI_SSID "Barlog"

#define WIFI_PASSWORD "Barlogu101"

// --- Time Settings ---

// Offset for Eastern European Time (EET): UTC + 2 hours = 7200 seconds

const long utcOffsetInSeconds = 7200;

// Initialize NTP Client to get time from the internet

WiFiUDP ntpUDP;

NTPClient timeClient(ntpUDP, "pool.ntp.org", utcOffsetInSeconds);

// Initialize RFID and Servo objects

MFRC522 mfrc522(SS_PIN, RST_PIN);

MFRC522::MIFARE_Key key;

MFRC522::StatusCode status;

Servo myservo;

// RFID Memory Block Configuration

// We are reading the User Name stored in Block 2 of the card

int blockNum = 2;

byte bufferLen = 18;

byte readBlockData[18]; // Buffer to store data read from the card

// Google Apps Script Web App URL (for logging data)

const String sheet_url = "https://script.google.com/macros/s/AKfycbx5AoK0nUO4Hl1oC-72DRaM7ljwO7O1MuxojZpNuLdddfSmVT6geBGqRU62YBATaLCkA/exec?name=";

void setup()

{

// 1. Initialize Servo and lock the door initially

myservo.attach(SERVO_PIN);

```

```

myservo.write(0); // 0 degrees = Locked

// 2. Connect to WiFi

// The system must hang here until connected because it relies on the internet

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

while (WiFi.status() != WL_CONNECTED){

    delay(500); // Wait 500ms before checking again

}

// 3. Start the Time Client (NTP)

timeClient.begin();

// 4. Initialize Peripherals

pinMode(BUZZER, OUTPUT);

SPI.begin();    // Init SPI bus

mfrc522.PCD_Init(); // Init RFID reader

}

void loop()

{

    // Updates the time from the server every loop iteration

    timeClient.update();

    // --- RFID Detection ---

    // If no new card is present, exit the loop function and try again

    if ( ! mfrc522.PICC_IsNewCardPresent()) { delay(500); return; }

    // If we can't read the serial data, exit

    if ( ! mfrc522.PICC_ReadCardSerial()) {return;}

    // Read the custom name stored in Block 2

    ReadDataFromBlock(blockNum, readBlockData);

    // --- Data Processing ---

    // Convert the byte array from the card into a readable String

    String rfid_name = "";

    for (int j = 0; j < 16; j++){

        if (readBlockData[j] != 0x00) { // Ignore empty bytes

            rfid_name += (char)readBlockData[j];

        }

    }

    rfid_name.trim(); // Remove any extra whitespace

    // Audio feedback: Short beep to acknowledge scan

    digitalWrite(BUZZER, HIGH); delay(100); digitalWrite(BUZZER, LOW);

```

```

// --- Access Control Logic ---

bool openDoor = false;

int currentHour = timeClient.getHours(); // Get current hour (0-23)

// Case 1: Administrator
if (rfid_name == "Boss") {

    openDoor = true; // Unlimited access

}

// Case 2: Restricted Worker
else if (rfid_name == "Worker1") {

    // Time Logic: Access allowed only between 17:00 and 19:00 (5 PM - 7 PM)
    if (currentHour >= 17 && currentHour < 19) {

        openDoor = true;

    } else {

        openDoor = false; // Outside of shift hours

    }

}

// Case 3: Unknown or unauthorized card
else {

    openDoor = false;

}

// --- Actuation ---

if (openDoor) {

    // Unlock sequence

    myservo.write(180); // Open door

    digitalWrite(BUZZER, HIGH); delay(100); digitalWrite(BUZZER, LOW); // Success beep

    delay(3000); // Keep door open for 3 seconds

    myservo.write(0); // Lock door again

} else {

    // Access Denied sequence: Long beep

    digitalWrite(BUZZER, HIGH); delay(800); digitalWrite(BUZZER, LOW);

}

// --- Cloud Logging ---

// Prepare status string based on the decision
String statusToSend = openDoor ? "Granted" : "Denied";

// Send the data (Name + Status) to Google Sheets
SendToGoogleSheets(rfid_name, statusToSend);

```

```

// Stop encryption on the card and wait before next scan

delay(2000);

mfr522.PICC_HaltA();

mfr522.PCD_StopCrypto1();

}

// Helper Function: Reads data from a specific block on the RFID card

void ReadDataFromBlock(int blockNum, byte readBlockData[])

{

// Prepare the default key (0xFFFFFFFFFFFF)

for (byte i = 0; i < 6; i++) key.keyByte[i] = 0xFF;

// Authenticate using Key A

status = mfr522.PCD_Authenticate(MFRC522::PICC_CMD_MF_AUTH_KEY_A, blockNum, &key, &(mfr522.uid));

if (status != MFRC522::STATUS_OK) return; // Auth failed


// Read the data

status = mfr522.MIFARE_Read(blockNum, readBlockData, &bufferLen);

}

// Helper Function: Sends an HTTPS GET request to Google Scripts

void SendToGoogleSheets(String name, String status) {

if (WiFi.status() == WL_CONNECTED) {

// Use WiFiClientSecure for HTTPS

std::unique_ptr<BearSSL::WiFiClientSecure>client(new BearSSL::WiFiClientSecure);

// IMPORTANT: Skip SSL certificate validation.

// This is less secure but necessary for simple ESP8266 projects

// to avoid managing complex certificates.

client->setInsecure();

// Construct the full URL with parameters

String url = sheet_url + name + "&sts=" + status;

HTTPClient https;

if (https.begin(*client, url)){

// Send the GET request to trigger the script

https.GET();

https.end(); // Close connection

}

}

}

```

e) Google Apps Script

```
var ss = SpreadsheetApp.openById('1yNHPyoRZM-1ydtl1PFik4ZdhlyTHesxhxxBORKn_dY');

var sheet = ss.getSheetByName('Sheet1');

var timezone = "Europe/Bucharest";

function doGet(e){

  if (e.parameter == 'undefined') { return ContentService.createTextOutput("Received data is undefined"); }

  var Curr_Date = new Date();

  var Curr_Time = Utilities.formatDate(Curr_Date, timezone, 'HH:mm:ss');

  var name = stripQuotes(e.parameter.name);

  var accessStatus = stripQuotes(e.parameter.sts);

  var nextRow = sheet.getLastRow() + 1;

  sheet.getRange("A" + nextRow).setValue(Curr_Date);

  sheet.getRange("B" + nextRow).setValue(Curr_Time);

  sheet.getRange("C" + nextRow).setValue(name);

  var rowRange = sheet.getRange("A" + nextRow + ":" + "C" + nextRow);

  if (accessStatus == "Granted") {

    rowRange.setBackground("#d9ead3");

    rowRange.setFontColor("#274e13");

    rowRange.setFontWeight("bold");

  } else {

    rowRange.setBackground("#f4cccc");

    rowRange.setFontColor("#cc0000");

    rowRange.setFontWeight("bold");

  }

  return ContentService.createTextOutput("Attendance recorded.");

}

function stripQuotes( value ) { return value.toString().replace(/^["]|["]$/g, ""); }
```

4. CONCLUSIONS

Working on this project helped me understand how hardware and cloud software can work together. The system successfully meets all the requirements: it reads cards, enforces time limits for the worker profile, and logs everything to the cloud without issues.

Results:

- The time-restriction logic works perfectly; "Worker1" cannot open the door outside of shift hours.
- The Google Sheet updates almost instantly (within 1-2 seconds of the card scan).
- The visual feedback (buzzer sounds and spreadsheet colors) makes the system very easy to monitor.

Bibliography

1. [ESP8266 Datasheet](#)
2. [RFID RC522 Manual](#)
3. [Spreadsheets](#)
4. [Helpful Ideas](#)